

Matthew Tran

Public-Key Cryptography with RSA

In this lab, I gained hands on experience implementing the RSA public key cryptosystem using C and the OpenSSL Big Number library. RSA is foundational to modern secure communication, enabling encryption, decryption, and digital signatures through mathematical operations on large prime numbers. I learned how to generate keys, encrypt and decrypt messages, sign data, and verify signatures, all while working with numbers far larger than standard data types can handle.

Tools Used

- C Programming Language
- OpenSSL Big Number Library
- Ubuntu
- GCC Compiler

Skills Gained

- Working with arbitrary precision arithmetic
- Implementing RSA key generation, encryption, and decryption
- Generating and verifying digital signatures
- Understanding cryptographic integrity and the avalanche effect

Task 1 Big Number Arithmetic Operations

1. Created task1.c and included necessary headers (stdio.h, openssl/bn.h).
2. Generated a 256 bit random prime for a , assigned a large decimal value to b , and generated a 256 bit random number for n .
3. Computed:
 $a * b$
 $a^b \bmod n$

4. Used a custom function printBN_dec() to print results in decimal format.
5. Compiled with: gcc task1.c -lcrypto -o task1

```
#include <stdio.h>
#include <openssl/bn.h>

#include <stdio.h>
#include <openssl/bn.h>

void printBN_dec(const char *msg, const BIGNUM *a) {
    char *str = BN_bn2dec(a);
    if (!str) {
        printf("%s <error converting to decimal>\n", msg);
        return;
    }
    printf("%s %s\n", msg, str);
    OPENSSL_free(str);
}

int main() {
    BN_CTX *ctx = BN_CTX_new();
    if (!ctx) return 1;

    BIGNUM *a = BN_new();
    BIGNUM *b = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *prod = BN_new();
    BIGNUM *modres = BN_new();

    if (!BN_generate_prime_ex(a, 256, 1, NULL, NULL, NULL)) {
        fprintf(stderr, "Error generating prime a\n");
        return 1;
    }

    BN_dec2bn(&b, "273489463796838501848592769467194369268");

    if (!BN_rand(n, 256, 0, 0)) {
        fprintf(stderr, "Error generating random n\n");
        return 1;
    }

    BN_mul(prod, a, b, ctx);

    BN_mod_exp(modres, a, b, n, ctx);

    printBN_dec("a (decimal) = ", a);
    printBN_dec("b (decimal) = ", b);
    printBN_dec("n (decimal) = ", n);
    printBN_dec("a * b = ", prod);
    printBN_dec("(a^b) mod n = ", modres);

    BN_free(a);
    BN_free(b);
    BN_free(n);
    BN_free(prod);
    BN_free(modres);
    BN_CTX_free(ctx);
    return 0;
}
```

Output :

```
matt@matt-VirtualBox:~$ gcc task1.c -lcrypto -o task1
matt@matt-VirtualBox:~$ ./task1
a (decimal) = 1118142979498983192446886887963128923713248499618744423767934123
08139275748987
b (decimal) = 273489463796838501848592769467194369268
n (decimal) = 6943796066278881151614276311976507280567051389221326040863934784
9572102117638
a * b = 3058003239113762989848162967376916470886322299166751299571248231088000
0518418755055787251657613024271249309754931516
(a^b) mod n = 5483738345826756132226958006351449056336136851301133759855379383
0425441263685
```

A and n are randomly generated, so each run will produce different results. The output shows that OpenSSL's Bignum can handle extremely large multiplication and modular operations

correctly.

Task 2 RSA Private Key Derivation

```
matt@matt-VirtualBox: ~  
#include <stdio.h>  
#include <openssl/bn.h>  
  
void printBN_hex(const char *msg, const BIGNUM *a) {  
    char *str = BN_bn2hex(a);  
    if (!str) {  
        printf("%s <error converting to hex>\n", msg);  
        return;  
    }  
    printf("%s %s\n", msg, str);  
    OPENSSL_free(str);  
}  
  
int main() {  
    BN_CTX *ctx = BN_CTX_new();  
    BIGNUM *p = BN_new();  
    BIGNUM *q = BN_new();  
    BIGNUM *e = BN_new();  
    BIGNUM *n = BN_new();  
    BIGNUM *phi = BN_new();  
    BIGNUM *p_minus1 = BN_new();  
    BIGNUM *q_minus1 = BN_new();  
    BIGNUM *d = NULL;  
  
    BN_hex2bn(&p, "F7E75FDC469067FFDC4E847C51F452DF");  
    BN_hex2bn(&q, "E85CED54AF57E53E092113E62F436F4F");  
    BN_hex2bn(&e, "0D88C3");  
  
    BN_mul(n, p, q, ctx);  
  
    BN_sub(p_minus1, p, BN_value_one());  
    BN_sub(q_minus1, q, BN_value_one());  
    BN_mul(phi, p_minus1, q_minus1, ctx);  
  
    d = BN_mod_inverse(NULL, e, phi, ctx);  
    if (!d) {  
        fprintf(stderr, "Error computing modular inverse (d)\n");  
        return 1;  
    }  
  
    printBN_hex("p =", p);  
    printBN_hex("q =", q);  
    printBN_hex("e =", e);  
    printBN_hex("n = p*q =", n);  
    printBN_hex("d = e^{-1} mod phi =", d);  
  
    BN_free(p); BN_free(q); BN_free(e); BN_free(n); BN_free(phi);  
    BN_free(p_minus1); BN_free(q_minus1); BN_free(d);  
    BN_CTX_free(ctx);  
  
    return 0;  
}
```

1. Created task2.c and included OpenSSL headers.
2. Assigned given hexadecimal values to p, q, and e.
3. Computed:
$$n = p * q$$
$$\phi(n) = (p - 1) * (q - 1)$$
$$d = e^{-1} \bmod \phi(n)$$

4. Printed results in hexadecimal using `printBN_hex()`.

Output:

```
matt@matt-VirtualBox:~$ gcc task2.c -lcrypto -o task2
matt@matt-VirtualBox:~$ ./task2
p = F7E75FDC469067FFDC4E847C51F452DF
q = E85CED54AF57E53E092113E62F436F4F
e = 0D88C3
n = p*q = E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1
d = e^{-1} mod phi = 3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B
496AEB
```

The computed n and d represent the RA modulus and private exponent. The value of d is the modular inverse of e with respect to $\phi(n)$, completing the key generation step.

Task 3 RSA Message Encryption

1. Created `task3.c` and converted the message "A top secret!" to hex using Python.
2. Loaded the given public key (n , e) and plaintext M as `BIGNUM`s.
3. Encrypted using: $C = M^e \bmod n$
4. Printed ciphertext in hex.

```
// task3
#include <stdio.h>
#include <openssl/bn.h>

void printBN_hex(const char *msg, const BIGNUM *a) {
    char *str = BN_bn2hex(a);
    if (!str) {
        printf("%s <error converting to hex>\n", msg);
        return;
    }
    printf("%s %s\n", msg, str);
    OPENSSL_free(str);
}

int main() {
    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *n = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *m = BN_new();
    BIGNUM *c = BN_new();

    BN_hex2bn(&n, "DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDD3A4D0CB81629242FB1A5");
    BN_hex2bn(&e, "010001");
    BN_hex2bn(&m, "4120746F702073656372657421");

    BN_mod_exp(c, m, e, n, ctx);

    printBN_hex("Plaintext M (hex) =", m);
    printBN_hex("Ciphertext C (hex) =", c);

    BN_free(n); BN_free(e); BN_free(m); BN_free(c);
    BN_CTX_free(ctx);
    return 0;
}
```

Output:

```
matt@matt-VirtualBox:~$ gcc task3.c -lcrypto -o task3
matt@matt-VirtualBox:~$ ./task3
Plaintext M (hex) = 4120746F702073656372657421
Ciphertext C (hex) = 6FB078DA550B2650832661E14F4F8D2CFAEF475A0DF3A75CACDC5DE5CF
C5FADC
```

The ciphertext is the encrypted form of the original message created using the public key.

Task 4 RSA Message Decryption

1. Created task4.c and loaded the given n, d, and ciphertext C.
2. Decrypted using:
3. $M = C^d \bmod n$
4. Printed the decrypted hex and converted it to ASCII using Python.

```
// task4.c
#include <stdio.h>
#include <openssl/bn.h>
#include <openssl/err.h>

void printBN_hex(const char *msg, const BIGNUM *a) {
    char *str = BN_bn2hex(a);
    if (!str) {
        printf("%s <error converting to hex>\n", msg);
        return;
    }
    printf("%s %s\n", msg, str);
    OPENSSL_free(str);
}

int main() {
    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *n = BN_new();
    BIGNUM *d = BN_new();
    BIGNUM *c = BN_new();
    BIGNUM *m = BN_new();

    BN_hex2bn(&n, "DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
    BN_hex2bn(&d, "74D806F9F3A62BAE331FFE3F0A68AFE35B3D2E4794148AACBC26AA381CD7D30D");
    BN_hex2bn(&c, "8C0F971DF2F3672B28811407E2DABBE1DA0FEBBDFC7DCB67396567EA1E2493F");
```

```

BN_mod_exp(m, c, d, n, ctx);

printBN_hex("Ciphertext C (hex) =", c);
printBN_hex("Decrypted M (hex) =", m);

char *m_hex = BN_bn2hex(m); // hex string
if (m_hex) {

    OPENSSL_free(m_hex);
}

BN_free(n); BN_free(d); BN_free(c); BN_free(m);
BN_CTX_free(ctx);
return 0;
}

```

Output:

```

matt@matt-VirtualBox:~$ gcc task4.c -lcrypto -o task4
matt@matt-VirtualBox:~$ ./task4
Ciphertext C (hex) = 8C0F971DF2F3672B28811407E2DABBE1DA0FEBBBBDFC7DCB67396567EA1
E2493F
Decrypted M (hex) = 50617373776F72642069732064656573

```

The decrypted message shows a successful recovery of the plaintext. The output verifies the correct uses of modular exponentiation with the private key.

Task 5 Digital Signature Generation

1. Created task5.c with a helper function `bn_from_ascii()` to convert strings to BIGNUMs.
2. Signed two messages:
 "I owe you \$2000."
 "I owe you \$3000."
3. Used private key `d` to compute:

$$S = M^d \bmod n$$
4. Printed both signatures.

```

// task5.c
#include <stdio.h>
#include <string.h>
#include <openssl/bn.h>

BIGNUM* bn_from_ascii(const char *s) {
    size_t len = strlen(s);
    char *hex = malloc(len * 2 + 1);
    if (!hex) return NULL;
    unsigned char *us = (unsigned char*) s;
    for (size_t i = 0; i < len; ++i) {
        sprintf(hex + 2*i, "%02X", us[i]);
    }
    hex[2*len] = 0;
    BIGNUM *bn = BN_new();
    BN_hex2bn(&bn, hex);
    free(hex);
    return bn;
}

void printBN_hex(const char *msg, const BIGNUM *a) {
    char *str = BN_bn2hex(a);
    if (!str) {
        printf("%s <error converting to hex>\n", msg);
        return;
    }
    printf("%s %s\n", msg, str);
    OPENSSL_free(str);
}

int main() {
    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *n = BN_new();
    BIGNUM *d = BN_new();

    BN_hex2bn(&n, "DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
    BN_hex2bn(&d, "74D806F9F3A62BAE331FFE3F0A68AFE35B3D2E4794148AACBC26AA381CD7D30D");

    const char *M1 = "I owe you $2000.";
    const char *M2 = "I owe you $3000.";

    BIGNUM *m1 = bn_from_ascii(M1);
    BIGNUM *m2 = bn_from_ascii(M2);
    BIGNUM *s1 = BN_new();
    BIGNUM *s2 = BN_new();

    BN_mod_exp(s1, m1, d, n, ctx);
    BN_mod_exp(s2, m2, d, n, ctx);

    printBN_hex("M1 (hex) =", m1);
    printBN_hex("Signature S1 (hex) =", s1);
    printBN_hex("M2 (hex) =", m2);
    printBN_hex("Signature S2 (hex) =", s2);

    BN_free(n); BN_free(d);
    BN_free(m1); BN_free(m2); BN_free(s1); BN_free(s2);
    BN_CTX_free(ctx);
    return 0;
}

```

Output:

```

matt@matt-VirtualBox:~$ gcc task5.c -lcrypto -o task5
matt@matt-VirtualBox:~$ ./task5
M1 (hex) = 49206F776520796F752024323030302E
Signature S1 (hex) = 55A4E7F17F04CCFE2766E1EB32ADDBA890BBE92A6FBE2D785ED6E73CCB35E4CB
M2 (hex) = 49206F776520796F752024333030302E
Signature S2 (hex) = BCC20FB7568E5D48E434C387C06A6025E90D29D848AF9C3EBAC0135D99305822

```

When there is a small change in the message, the results show me a different signature. This is probably showing the avalanche effect in cryptographic signatures when changing a small minor detail can change the outputs vastly. This ensures that the signature will always have a uniqueness and reinforce its security.

Task 6 Digital Signature Verification

1. Created task6.c with the same bn_from_ascii() helper.
2. Loaded given n, e, and signature S.
3. Verified by computing: $M' = S^e \bmod n$
4. Compared M' with expected message M.
5. Repeated with a corrupted signature (last byte changed from 2F to 3F).

```
#include <stdio.h>
#include <string.h>
#include <openssl/bn.h>

BIGNUM* bn_from_ascii(const char *s) {
    size_t len = strlen(s);
    char *hex = malloc(len * 2 + 1);
    if (!hex) return NULL;

    for (size_t i = 0; i < len; ++i)
        sprintf(hex + 2 * i, "%02X", (unsigned char)s[i]);
    hex[2 * len] = '\0';

    BIGNUM *bn = BN_new();
    BN_hex2bn(&bn, hex);
    free(hex);
    return bn;
}

void printBN_hex(const char *msg, const BIGNUM *a) {
    char *str = BN_bn2hex(a);
    if (str == NULL) {
        printf("%s <error converting to hex>\n", msg);
        return;
    }
    printf("%s %s\n", msg, str);
}
```

```

OPENSSL_free(str);
}

int main() {
    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *n = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *S = BN_new();
    BIGNUM *m_from_S = BN_new();

    BN_hex2bn(&n, "AE1CD4DC432798D933779FBD46C6E1247F0CF1233595113AA51B450F18116115");
    BN_hex2bn(&e, "010001");
    BN_hex2bn(&S, "643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CDBD6802F");

    BN_mod_exp(m_from_S, S, e, n, ctx);
    printBN_hex("S^e mod n (hex) =", m_from_S);

    const char *M = "Launch a missile.";
    BIGNUM *m_expected = bn_from_ascii(M);
    printBN_hex("M (hex) expected =", m_expected);

    if (BN_cmp(m_from_S, m_expected) == 0)
        printf("Signature verification PASSED: S is a valid signature for M\n");
    else
        printf("Signature verification FAILED: S is NOT a valid signature for M\n");

    BIGNUM *S_corr = BN_new();
    BIGNUM *m_corr = BN_new();
    BN_hex2bn(&S_corr, "643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CDBD6803F");
    BN_mod_exp(m_corr, S_corr, e, n, ctx);
    printBN_hex("Corrupted S^e mod n (hex) =", m_corr);

    if (BN_cmp(m_corr, m_expected) == 0)
        printf("Corrupted signature STILL verifies (unexpected)\n");
    else
        printf("Corrupted signature does NOT verify (expected): verification fails\n");

    BN_free(n);
    BN_free(e);
    BN_free(S);
    BN_free(m_from_S);

    BN_free(m_expected);
    BN_free(S_corr);
    BN_free(m_corr);
    BN_CTX_free(ctx);

    return 0;
}

```

Output:

```

matt@matt-VirtualBox:~$ gcc task6.c -lcrypto -o task6
matt@matt-VirtualBox:~$ ./task6
S^e mod n (hex) = 4C61756E63682061206D697373696C652E
M (hex) expected = 4C61756E63682061206D697373696C652E
Signature verification PASSED: S is a valid signature for M
Corrupted S^e mod n (hex) = 91471927C80DF1E42C154FB4638CE8BC726D3D66C83A4EB6B7BE0203B41AC294
Corrupted signature does NOT verify (expected): verification fails

```

The good signature verifies successfully (M' matches M), while the corrupted signature (one bit

changed) fails verification. This shows that digital signatures are extremely responsive to any modification. Even a single bit of change makes the signature invalid.